Les tests unitaires avec PHPUnit

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Contexte 3

II - Introduction et mise en place des tests unitaires 4

A. Les fondamentaux des tests unitaires ... 4
1. Pourquoi tester son code et principes des tests unitaires ... 4
2. Introduction à PHPUnit et son intégration avec Symfony ... 7
3. Structure d'un projet Symfony avec tests unitaires ... 10

B. Configuration et premier test ... 11
1. Installation et configuration de PHPUnit ... 11
2. Exercice pratique : mise en place de la structure de test ... 12

III - Techniques avancées et bonnes pratiques 14

A. Stratégies efficaces de test ... 14
1. Approches et patterns de test unitaire ... 14
2. Exercice pratique : rédaction d'un premier test unitaire ... 15

B. Tests avancés et intégration dans le workflow ... 16
1. Test d'exceptions et de cas limites ... 16
2. Intégration des tests dans le workflow de développement ... 18
3. Exercice pratique : tester une exception PHP ... 19

IV - Synthèse 21

V - Auto-évaluation 24

A. Exercice ... 24

B. Test ... 24

Solutions des exercices 27

Tous droits réservés à STUDI - Reproduction interdite

I Contexte

Les tests unitaires sont une pratique essentielle dans le développement professionnel d'applications. Des entreprises comme Doctolib, BlaBlaCar ou Société Générale s'appuient sur les tests automatisés pour garantir la qualité de leur code et prévenir les régressions. Symfony intègre nativement PHPUnit, un framework de test robuste, qui permet aux développeurs de vérifier que chaque composant de leur application fonctionne correctement de manière isolée.

Dans cette leçon, nous découvrirons comment les équipes de développement françaises implémentent les tests unitaires dans leurs projets Symfony, de la configuration initiale à l'écriture de tests avancés, en passant par les bonnes pratiques pour maintenir une suite de tests efficace et performante.

Tous droits réservés à STUDI - Reproduction interdite

II Introduction et mise en place des tests unitaires

A. Les fondamentaux des tests unitaires

1. Pourquoi tester son code et principes des tests unitaires

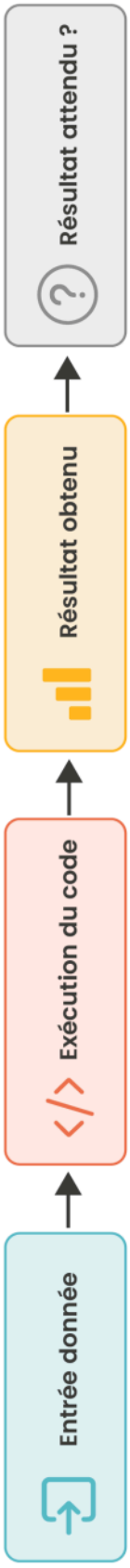
Avant même de découvrir les outils de test, il est important de comprendre pourquoi les tests occupent aujourd'hui une place centrale dans le développement logiciel. Un test permet de vérifier automatiquement qu'un comportement fonctionne comme prévu et continue de fonctionner au fil du temps. L'intérêt est immédiat : plus une anomalie est détectée tôt, moins elle coûte cher à corriger. Les tests servent également de filet de sécurité lors des évolutions du projet. Lorsqu'un développeur modifie une fonctionnalité ou refactorise du code, il peut exécuter la suite de tests pour vérifier que les comportements existants n'ont pas été cassés involontairement.

Au-delà de la détection d'erreurs, les tests jouent aussi un rôle de documentation. Ils décrivent de manière concrète ce qu'une méthode ou une classe est censée faire. Lorsqu'un nouveau développeur rejoint un projet, la lecture des tests lui permet souvent de comprendre rapidement les règles métier et les comportements attendus.

Dans l'univers PHP, PHPUnit est l'outil de référence pour écrire des tests unitaires. Son objectif est de vérifier le comportement d'une unité de code isolée, généralement une méthode, une classe ou un scénario d'exécution très précis.

Tous droits réservés à STUDI - Reproduction interdite

Un test unitaire répond donc à une question simple :



Si le résultat obtenu correspond exactement au résultat attendu, le test réussit. Dans le cas contraire, le test échoue et signale un problème.

Tous droits réservés à STUDI - Reproduction interdite

Pour être réellement utile, un test unitaire doit respecter plusieurs principes fondamentaux :

1. **La rapidité**

   Une suite de tests doit pouvoir être exécutée fréquemment sans ralentir le travail de l'équipe. Si chaque exécution prend plusieurs minutes, les développeurs auront naturellement tendance à lancer les tests moins souvent.

2. **Le déterminisme**

   Un même test exécuté plusieurs fois dans les mêmes conditions doit toujours produire le même résultat. Un test qui réussit parfois et échoue parfois sans modification du code devient rapidement inutilisable.

3. **L'indépendance**

   Chaque test doit pouvoir s'exécuter seul, sans dépendre d'un autre test, d'un ordre d'exécution particulier ou d'un état laissé par une opération précédente. Cela garantit des résultats fiables et facilite l'identification des erreurs lorsqu'un test échoue.

Cette exigence d'indépendance implique notamment d'éviter les dépendances directes à des ressources externes comme une base de données, un service web ou un système de fichiers lorsque l'on réalise un véritable test unitaire.

L'objectif est de tester uniquement la logique de l'unité concernée.

Exemple

Prenons un exemple simple. Imaginons une méthode chargée de calculer une remise :

```php
public function calculateDiscount(float $price): float
{
    return $price * 0.9;
}
```

Le rôle du test est alors de vérifier que, pour une entrée donnée, la méthode retourne exactement le résultat attendu.

Le raisonnement devient très explicite :

Prix : 100 € → calculateDiscount() → 90 €

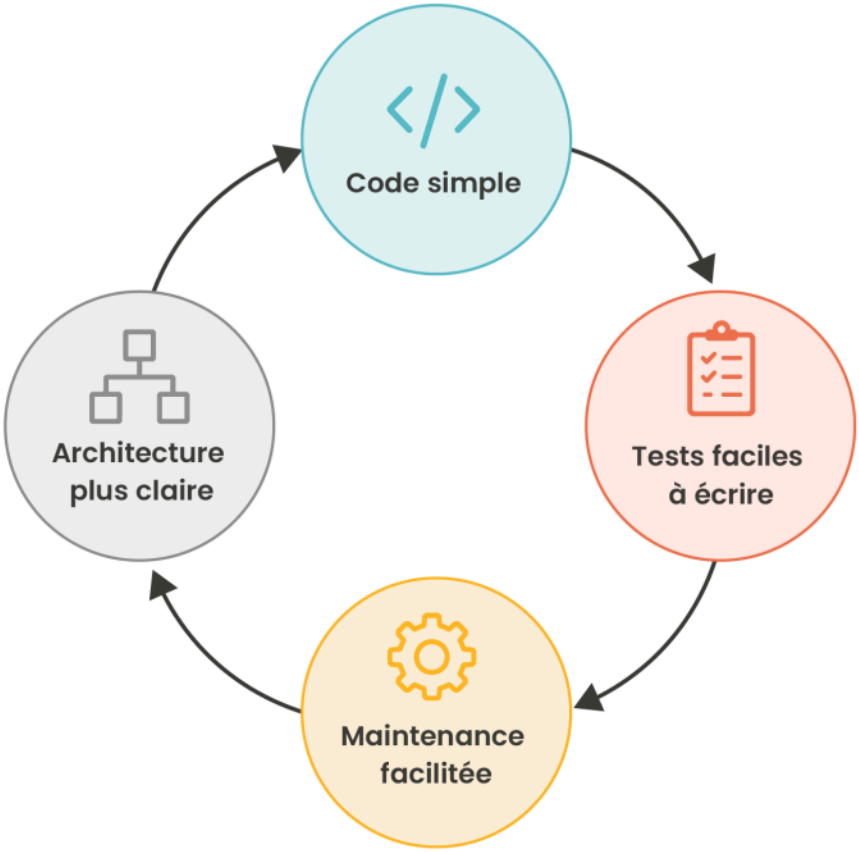
Le test décrit ainsi un comportement métier précis et vérifiable.

Cette approche apporte un autre bénéfice souvent sous-estimé : elle pousse naturellement à produire un code mieux conçu. En effet, lorsqu'une classe devient difficile à tester, cela révèle souvent un problème de conception. Une classe qui dépend de nombreux services, manipule plusieurs responsabilités ou accède directement à des ressources externes sera généralement plus complexe à tester.

À l'inverse, une classe possédant une responsabilité claire et des dépendances explicites sera beaucoup plus simple à valider.

Tous droits réservés à STUDI - Reproduction interdite

On observe alors un cercle vertueux :



C'est pour cette raison que les tests unitaires ne servent pas uniquement à vérifier le fonctionnement du code. Ils influencent également la qualité globale de l'architecture. Lorsqu'un développeur adopte une démarche orientée test, il est naturellement encouragé à découper les responsabilités, à expliciter les dépendances et à limiter le couplage entre les composants.

2. Introduction à PHPUnit et son intégration avec Symfony

Maintenant que nous avons vu pourquoi les tests unitaires sont importants, il est temps de découvrir l'outil qui permet de les exécuter dans l'écosystème PHP : PHPUnit.

PHPUnit est le framework de tests le plus utilisé dans le monde PHP. Son rôle est de fournir toutes les briques nécessaires pour écrire, organiser et exécuter automatiquement des tests. Symfony s'appuie naturellement sur cet outil et propose même une intégration prête à l'emploi qui simplifie considérablement son utilisation.

Dans un projet Symfony, PHPUnit s'appuie généralement sur une structure standard qui facilite l'organisation des tests.

On retrouve notamment :

`tests/`

`phpunit.dist.xml`

`tests/bootstrap.php`

Le dossier tests/ accueille l'ensemble des classes de test, tandis que les fichiers de configuration permettent à PHPUnit de connaître l'environnement dans lequel les tests doivent s'exécuter.

Cette organisation apporte un cadre cohérent pour structurer les campagnes de tests et séparer clairement le code applicatif du code de vérification.

Tous droits réservés à STUDI - Reproduction interdite

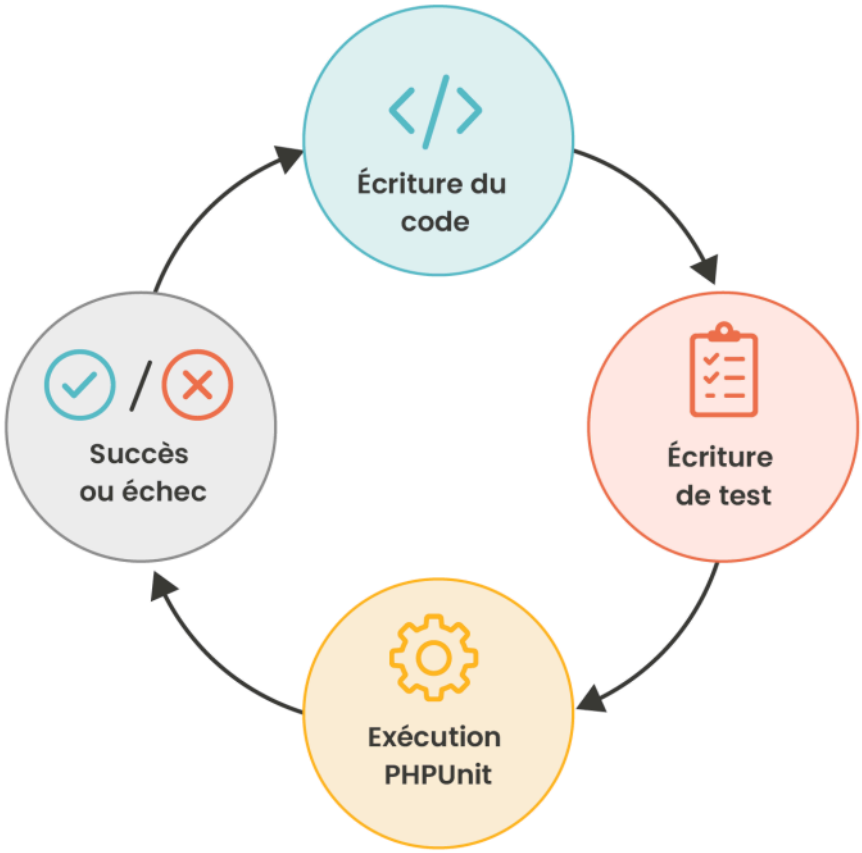
PHPUnit repose sur un principe simple : chaque test exécute un scénario précis puis compare le résultat obtenu avec le résultat attendu. Si les deux correspondent, le test est considéré comme réussi. Dans le cas contraire, PHPUnit signale immédiatement l'échec afin de faciliter l'identification du problème.

Au-delà de l'exécution des tests, PHPUnit fournit également de nombreux outils destinés à simplifier le travail des développeurs : assertions, jeux de données, mocks, rapports détaillés ou encore couverture de code.

Selon les versions du projet et des outils utilisés, le fichier de configuration principal peut s'appeler `phpunit.dist.xml` ou `phpunit.xml.dist`.

Cette différence est essentiellement historique et n'a généralement pas d'impact sur l'utilisation de PHPUnit. Elle explique toutefois pourquoi les deux variantes peuvent encore être rencontrées dans des projets existants.

Une fois les premiers tests écrits, PHPUnit fournit également un retour immédiat sur leur état. Le cycle de travail devient alors très simple :



Cette boucle rapide favorise les corrections précoces et réduit le risque d'introduire des régressions.

Cependant, Symfony ne se limite pas à l'intégration standard de PHPUnit : le framework fournit également un outil complémentaire particulièrement utile : le PHPUnit Bridge. Ce composant ajoute plusieurs fonctionnalités destinées à améliorer l'expérience de test dans les projets Symfony.

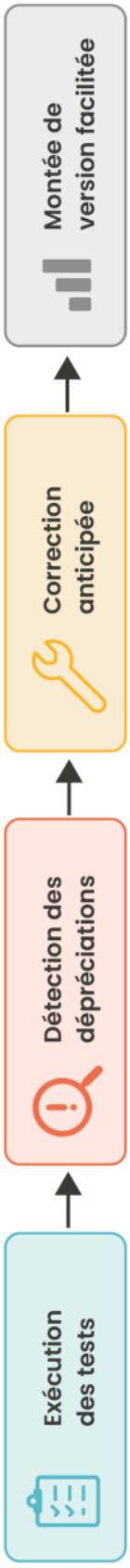
Parmi ses apports les plus intéressants figure le script :

```
vendor/bin/simple-phpunit
```

Ce script encapsule PHPUnit et enrichit ses capacités avec des fonctionnalités spécifiques à Symfony. L'un des avantages majeurs concerne la gestion des dépréciations. Lorsqu'une future version de Symfony prévoit la suppression d'une fonctionnalité, le Bridge peut signaler les éléments concernés directement pendant l'exécution des tests.

Tous droits réservés à STUDI - Reproduction interdite

Le flux devient alors :



Tous droits réservés à STUDI - Reproduction interdite

Cette capacité est particulièrement précieuse lors des migrations entre versions majeures du framework. Dans les projets de longue durée, il arrive toutefois que certaines dépréciations soient déjà connues mais ne puissent pas être corrigées immédiatement. Pour éviter qu'elles ne polluent constamment les rapports de tests, Symfony permet de générer une baseline de dépréciations grâce à la variable :

SYMFONY_DEPRECATIONS_HELPER

Le principe consiste à enregistrer l'état actuel des dépréciations connues afin de ne signaler ensuite que les nouvelles alertes. Cette approche est particulièrement utile dans les projets en maintenance ou lors des montées de version progressives. Ainsi, les équipes peuvent continuer à surveiller l'apparition de nouvelles dépréciations sans être noyées sous des avertissements déjà identifiés.

3. Structure d'un projet Symfony avec tests unitaires

Une fois PHPUnit installé, il est important d'adopter une organisation cohérente des fichiers de test. Une bonne structure facilite la navigation dans le projet, améliore la lisibilité et permet à toute l'équipe de retrouver rapidement les éléments qu'elle recherche.

Dans un projet Symfony classique, la séparation fondamentale consiste à distinguer clairement le code de production du code de test. Le dossier src/ contient les classes utilisées par l'application, tandis que le dossier tests/ regroupe les scénarios destinés à vérifier leur comportement.

On retrouve généralement l'organisation suivante :

```
src/
└── Service/
    └── PriceCalculator.php
tests/
└── Service/
    └── PriceCalculatorTest.php
phpunit.dist.xml
tests/bootstrap.php
```

Cette structure simple repose sur une idée essentielle : chaque élément métier possède un test facilement identifiable. Lorsqu'un développeur consulte une classe du dossier src/, il doit pouvoir retrouver rapidement le test correspondant dans le dossier tests/.

La relation est volontairement symétrique :

src/Service/PriceCalculator.php → tests/Service/PriceCalculatorTest.php

Cette structure présente plusieurs avantages. Tout d'abord, elle réduit le temps passé à rechercher les tests associés à une fonctionnalité. Ensuite, elle facilite la maintenance lorsque le projet grossit et que plusieurs développeurs travaillent simultanément sur le même code.

Pour un projet pédagogique ou une application de taille modérée, cette approche est généralement suffisante. Cependant, lorsque la suite de tests devient plus importante, il peut être utile de structurer davantage le dossier tests/.

Une pratique courante consiste à distinguer plusieurs catégories de tests :

```
tests/
├── Unit/
├── Integration/
└── Application/
```

Cette séparation permet de différencier les intentions de test.

Les tests unitaires vérifient une unité de code isolée.

Tous droits réservés à STUDI - Reproduction interdite

Les tests d'intégration contrôlent la collaboration entre plusieurs composants.

Les tests applicatifs ou fonctionnels valident le comportement global de l'application dans des scénarios proches de l'utilisation réelle.

L'intérêt est double : d'une part, la structure devient plus lisible pour les développeurs ; d'autre part, il devient plus simple d'exécuter uniquement une catégorie de tests lorsque cela est nécessaire.

Une bonne structure de projet n'est pas forcément la plus complexe, mais celle qui permet à chaque développeur de comprendre rapidement où se trouve le code métier, où se trouvent les tests associés et comment exécuter l'ensemble sans ambiguïté.

B. Configuration et premier test

1. Installation et configuration de PHPUnit

Avant d'écrire les premiers tests, il est nécessaire de mettre en place l'environnement de test du projet. Dans Symfony, cette étape est largement simplifiée grâce au package symfony/test-pack, qui installe PHPUnit ainsi que les outils couramment utilisés pour les tests.

L'installation s'effectue généralement avec la commande suivante :

```
composer require --dev symfony/test-pack
```

Une fois l'installation terminée, il est recommandé de vérifier immédiatement que l'environnement fonctionne correctement en exécutant la commande :

```
php bin/phpunit
```

Cette première exécution permet de s'assurer que PHPUnit est correctement installé et que la configuration du projet est opérationnelle.

Lors de l'installation, Symfony Flex génère automatiquement plusieurs fichiers de configuration essentiels. Parmi eux, on retrouve notamment phpunit.dist.xml, qui contient la configuration générale de PHPUnit, ainsi que tests/bootstrap.php, chargé de préparer l'environnement avant l'exécution des tests. Dans la majorité des projets, cette configuration générée automatiquement suffit largement pour commencer à écrire des tests unitaires sans avoir à effectuer de réglages supplémentaires.

Symfony exécute ses tests dans un environnement spécifique nommé test, distinct des environnements de développement et de production. Cette séparation permet d'utiliser des paramètres adaptés aux besoins des tests tout en garantissant leur reproductibilité. Lorsqu'un test nécessite le démarrage du framework, Symfony s'appuie généralement sur la variable KERNEL_CLASS, définie dans le fichier .env.test. Les paramètres peuvent ensuite être affinés dans .env.test.local si nécessaire.

L'ordre de chargement des variables d'environnement suit généralement la logique suivante :

.env → .env.test → .env.test.local

Une particularité importante mérite toutefois d'être soulignée : contrairement à l'environnement de développement, Symfony ne charge volontairement pas le fichier .env.local lors de l'exécution des tests. Cette décision évite qu'une configuration propre à un développeur influence les résultats obtenus. Ainsi, les tests exécutés sur une machine locale, sur un serveur d'intégration continue ou par un autre membre de l'équipe reposent sur les mêmes bases de configuration.

Il est également essentiel de distinguer les différents types de tests. Un véritable test unitaire n'a généralement besoin ni du kernel Symfony, ni du conteneur de services, ni d'une configuration complexe de l'environnement test. Dans ce cas, une simple instanciation de la classe à tester suffit :

```php
$calculator = new PriceCalculator();
$result = $calculator->calculate(100);
```

Dans cet exemple, aucun composant Symfony n'est démarré. Le test reste rapide, isolé et entièrement focalisé sur la logique métier de la classe.

Tous droits réservés à STUDI - Reproduction interdite

À l'inverse, lorsqu'un test doit interagir avec le conteneur de services, la configuration ou le kernel Symfony, on bascule vers des tests d'intégration ou des tests applicatifs. Il devient alors pertinent d'utiliser des outils comme KernelTestCase ou WebTestCase. Cette distinction est importante, car elle permet d'éviter une erreur fréquente chez les débutants, qui consiste à démarrer l'intégralité du framework pour tester une simple méthode métier. Plus un test reste isolé de son environnement, plus il est rapide, fiable et facile à maintenir.

2. Exercice pratique : mise en place de la structure de test

Après avoir découvert l'installation de PHPUnit et l'organisation générale d'un projet de tests, il est temps de mettre en pratique ces notions avec un premier exercice simple. L'objectif n'est pas encore de tester une logique métier complexe, mais de vérifier que toute la structure de test est correctement mise en place.

L'exercice consiste à créer une classe DiscountCalculator dans le dossier src/Service/, puis à créer sa classe de test associée DiscountCalculatorTest dans tests/Service/. Cette étape permet de reproduire l'organisation habituellement utilisée dans les projets Symfony et d'acquérir les bons réflexes de nommage et de structuration.

L'arborescence attendue est la suivante :

```
src/
└── Service/
    └── DiscountCalculator.php
tests/
└── Service/
    └── DiscountCalculatorTest.php
```

À ce stade, le contenu métier de la classe importe peu. L'objectif principal est de vérifier que PHPUnit détecte correctement le test et que l'environnement est capable de l'exécuter sans erreur.

Pour cela, la classe de test doit hériter de TestCase, utiliser le bon namespace et contenir au minimum une méthode de test. Une méthode setUp() sera également mise en place afin de préparer l'objet à tester avant chaque exécution.

Voici le squelette attendu :

```php
<?php

namespace App\Tests\Service;

use App\Service\DiscountCalculator;
use PHPUnit\Framework\TestCase;

final class DiscountCalculatorTest extends TestCase
{
    private DiscountCalculator $calculator;

    protected function setUp(): void
    {
        $this->calculator = new DiscountCalculator();
    }

    public function testExample(): void
    {
        $this->assertTrue(true);
    }
}
```

Même si ce premier test ne vérifie encore aucun comportement métier, il permet déjà de valider plusieurs éléments essentiels : la structure des dossiers, le chargement des classes, le bon fonctionnement de PHPUnit et l'absence d'erreur de configuration.

Tous droits réservés à STUDI - Reproduction interdite

Une notion importante apparaît également dans cet exemple : la méthode setUp().

PHPUnit exécute automatiquement cette méthode avant chaque test. Elle sert à préparer l'environnement nécessaire à l'exécution des scénarios de test. Cela évite de répéter plusieurs fois le même code d'initialisation dans chaque méthode. Chaque test est exécuté sur une nouvelle instance de la classe de test. Cela garantit l'indépendance des scénarios et évite qu'un test modifie l'état utilisé par un autre.

Dans notre exemple, l'instanciation du service est réalisée une seule fois dans setUp() :

```php
protected function setUp(): void
{
    $this->calculator = new DiscountCalculator();
}
```

Ainsi, toutes les futures méthodes de test pourront réutiliser directement $this->calculator sans avoir à répéter ce code.

À l'inverse, la méthode tearDown() est beaucoup moins utilisée dans les tests unitaires simples. Elle devient surtout utile lorsqu'il faut libérer des ressources externes, fermer des connexions ou nettoyer des objets particulièrement volumineux après l'exécution du test.

Une fois la structure créée, il reste à vérifier que PHPUnit détecte correctement le fichier. Pour cela, on peut exécuter directement ce test depuis la ligne de commande :

```
php bin/phpunit tests/Service/DiscountCalculatorTest.php
```

Cette exécution ciblée est particulièrement pratique pendant le développement, car elle évite de lancer toute la suite de tests à chaque modification.

À la fin de cet exercice, plusieurs points doivent être validés :

- La classe est placée dans le bon dossier
- Le namespace est correct
- La classe hérite bien de TestCase
- Le fichier se termine par Test.php
- La méthode setUp() est correctement exécutée
- PHPUnit détecte et lance le test sans erreur

Même si ce premier test est volontairement minimaliste, il constitue une étape importante. Avant de vérifier des comportements métier complexes, il faut s'assurer que toute l'infrastructure de test fonctionne correctement. Une fois cette base validée, il sera possible de remplacer progressivement assertTrue(true) par de véritables assertions métier et de construire une suite de tests réellement utile pour le projet.

Tous droits réservés à STUDI - Reproduction interdite

III Techniques avancées et bonnes pratiques

A. Stratégies efficaces de test

1. Approches et patterns de test unitaire

Une fois les premiers tests écrits, une nouvelle question apparaît rapidement : comment écrire des tests qui restent lisibles et maintenables dans le temps ? En effet, un test qui fonctionne techniquement n'est pas forcément un bon test. Lorsque les scénarios deviennent plus nombreux, il est important d'adopter une structure claire afin de comprendre immédiatement ce qui est testé.

Parmi les approches les plus répandues, le pattern AAA est probablement le plus utilisé. Son nom provient de trois étapes simples : Arrange, Act et Assert. Le principe consiste à séparer clairement la préparation du test, l'exécution du comportement testé et la vérification du résultat obtenu. Cette structure apporte immédiatement de la lisibilité. Lorsqu'un développeur ouvre un test, il peut identifier en quelques secondes ce qui est préparé, ce qui est exécuté et ce qui est réellement vérifié.

Prenons l'exemple d'un calculateur de remise :

```php
public function testApplyDiscount(): void
{
    // Arrange
    $calculator = new DiscountCalculator();

    // Act
    $result = $calculator->apply(100, 10);

    // Assert
    $this->assertEquals(90, $result);
}
```

Même sans connaître la classe testée, il est facile de comprendre le scénario : un calculateur est créé, une remise est appliquée, puis le résultat est comparé à la valeur attendue.

Une autre approche très répandue est le pattern Given / When / Then, issu des méthodes orientées comportement (BDD). Son objectif est de rendre les tests plus lisibles en les structurant comme un scénario fonctionnel. La partie Given décrit le contexte initial et les données de départ, When représente l'action ou l'événement déclencheur, puis Then exprime le résultat attendu. Cette approche facilite la compréhension des tests, aussi bien pour les développeurs que pour les autres acteurs du projet, en mettant davantage l'accent sur le comportement attendu de l'application que sur son implémentation technique. Cette approche est particulièrement appréciée lorsque les règles métier deviennent plus complexes.

Exemple

```
Given un client premium
When il commande un produit
Then une remise de 20 % est appliquée
```


Tous droits réservés à STUDI - Reproduction interdite

La structure ressemble davantage à un scénario fonctionnel qu'à une simple vérification technique. Cela facilite souvent la communication entre développeurs, testeurs et experts métier.

Il est important de comprendre que ces patterns ne modifient pas PHPUnit lui-même. Ils constituent avant tout des conventions d'écriture destinées à rendre les tests plus lisibles et plus cohérents.

Un bon test doit permettre de répondre rapidement à trois questions :

- Quel est le contexte ?
- Quelle action est exécutée ?
- Quel résultat est attendu ?

Lorsqu'un lecteur doit parcourir plusieurs dizaines de lignes pour comprendre cela, le test mérite généralement d'être simplifié.

Ces réflexions conduisent naturellement à deux approches souvent évoquées dans le monde des tests : le TDD et le BDD.

Le Test Driven Development (TDD) repose sur une idée simple : écrire d'abord un test qui échoue, puis développer le minimum de code nécessaire pour le faire réussir. Cette approche pousse naturellement à concevoir des composants simples, découplés et faciles à tester.

Le Behavior Driven Development (BDD) adopte une perspective légèrement différente. L'objectif n'est plus seulement de vérifier du code, mais de décrire un comportement attendu du point de vue métier. L'accent est mis sur le comportement observable plutôt que sur les détails techniques de l'implémentation.

Il ne faut toutefois pas considérer ces approches comme des obligations méthodologiques. Dans la pratique, de nombreux projets mélangent naturellement plusieurs styles. L'essentiel n'est pas d'appliquer rigoureusement un label, mais de produire des tests compréhensibles, fiables et faciles à maintenir.

2. Exercice pratique : rédaction d'un premier test unitaire

Après avoir découvert la structure d'un test et les principaux patterns d'écriture, il est temps de rédiger un premier test unitaire complet. L'objectif de cet exercice est de mettre en pratique les notions vues précédemment en testant une méthode simple de calcul de remise.

On suppose l'existence d'une méthode ayant la signature suivante :

`public function applyDiscount(int $priceCents, int $discountPercent): int`

Cette méthode reçoit un prix exprimé en centimes ainsi qu'un pourcentage de remise et retourne le montant final, également en centimes.

Pour ce premier exercice, nous allons couvrir un scénario très simple : un produit coûtant 10 000 centimes (100 €) avec une remise de 10 %. Le résultat attendu est donc 9 000 centimes.

Pour rédiger le test, nous pouvons naturellement nous appuyer sur le pattern AAA vu précédemment.

```php
public function testApplyDiscountReturnsPriceWithTenPercentReduction(): void
{
// Arrange
$price = 10000;
$discount = 10;
// Act
$result = $this->calculator->applyDiscount($price, $discount);
// Assert
$this->assertSame(9000, $result);
}
```

Tous droits réservés à STUDI - Reproduction interdite

Plusieurs éléments méritent ici d'être soulignés. Tout d'abord, le nom de la méthode de test est explicite. Un bon nom doit permettre de comprendre immédiatement ce qui est vérifié sans avoir à lire le contenu du test. Lorsque la suite de tests grandit, cette pratique devient extrêmement précieuse pour identifier rapidement les scénarios couverts.

L'autre point important concerne le choix de l'assertion. Dans cet exercice, l'utilisation de assertSame() est particulièrement pertinente. Contrairement à certaines assertions plus permissives, assertSame() vérifie à la fois la valeur et le type. Le test devient ainsi plus strict et détecte davantage d'erreurs potentielles. Ce choix est d'autant plus intéressant que la méthode manipule des entiers. L'utilisation de centimes plutôt que de nombres à virgule flottante constitue d'ailleurs une bonne pratique fréquente dans les applications manipulant des montants financiers.

Les calculs monétaires réalisés avec des flottants peuvent en effet produire des imprécisions difficiles à détecter. En travaillant directement avec des centimes sous forme d'entiers, ce risque disparaît.

Une fois le test rédigé, il est recommandé de l'exécuter seul afin de vérifier son bon fonctionnement.

php bin/phpunit tests/Service/DiscountCalculatorTest.php

Cette exécution ciblée permet d'obtenir un retour rapide sans lancer l'intégralité de la suite de tests.

B. Tests avancés et intégration dans le workflow

1. Test d'exceptions et de cas limites

Jusqu'à présent, les exemples présentés vérifiaient principalement des scénarios où tout se déroule correctement. Pourtant, un bon test unitaire ne se limite pas aux cas favorables. Il doit également vérifier la manière dont une méthode réagit face à des données invalides, à des valeurs extrêmes ou à des situations inattendues. Lorsqu'une méthode est conçue pour lever une exception dans certaines conditions, cette exception fait partie intégrante de son contrat. Elle mérite donc un test dédié au même titre qu'un résultat métier classique.

Prenons un exemple simple : une méthode de calcul qui refuse un prix négatif. Dans ce cas, le comportement attendu n'est pas un résultat numérique mais bien le déclenchement d'une exception.

PHPUnit fournit plusieurs outils pour tester ce type de scénario :

- expectException()
- expectExceptionCode()
- expectExceptionMessage()
- expectExceptionMessageMatches()

Un point méthodologique très important doit être retenu : les méthodes expectException*() doivent être appelées avant l'instruction censée déclencher l'erreur, idéalement juste avant celle-ci.

Exemple

Voici un exemple complet :

```php
<?php
namespace App\Tests\Service;

use App\Service\PriceCalculator;
use PHPUnit\Framework\TestCase;

final class PriceCalculatorExceptionTest extends TestCase
{
    public function testCalculateTtcThrowsExceptionWhenHtIsNegative(): void
    {
```


Tous droits réservés à STUDI - Reproduction interdite

```
        $calculator = new PriceCalculator();

        $this->expectException(\InvalidArgumentException::class);
        $this->expectExceptionMessage('Le prix HT ne peut pas être négatif.');

        $calculator->calculateTtc(-100, 20);
    }
}
```

Dans cet exemple, le test ne cherche pas à vérifier une valeur de retour. Son objectif est de confirmer que la méthode refuse correctement une donnée invalide en lançant l'exception attendue.

Au-delà des exceptions, il est utile d'apprendre à classer les scénarios de test selon trois grandes catégories.

La première correspond aux cas normaux. Il s'agit des situations les plus courantes, celles que les utilisateurs rencontrent au quotidien.

- Prix : 100 €
- TVA : 20 %
- Résultat attendu : 120 €

La deuxième catégorie concerne les cas limites. Ces scénarios se situent aux frontières du domaine métier et sont souvent responsables de nombreux bugs.

- Remise : 0 %
- Panier : 0 centime
- Remise : 100 %
- TVA : 0 %

Même si ces valeurs paraissent particulières, elles restent généralement valides et doivent produire un résultat cohérent.

Enfin, la troisième catégorie regroupe les cas exceptionnels, c'est-à-dire les situations qui doivent être refusées par l'application.

- Prix négatif
- Taux incohérent
- Valeur null interdite
- Format invalide

Dans ces situations, l'objectif n'est plus de calculer un résultat mais de garantir que le système protège correctement ses règles métier.

Une dernière nuance mérite également d'être connue concernant expectExceptionMessage().

$this->expectExceptionMessage(

'Le prix HT ne peut pas être négatif.'

);

Contrairement à ce que l'on pourrait penser, PHPUnit ne vérifie pas nécessairement une égalité stricte du message. La méthode vérifie que le texte attendu est bien présent dans le message réel. Cette souplesse permet de rendre les tests plus robustes face à de petites évolutions rédactionnelles sans remettre en cause le comportement métier testé.

Tous droits réservés à STUDI - Reproduction interdite

En pratique, une suite de tests efficace combine toujours ces trois approches : des scénarios normaux pour valider le fonctionnement attendu, des cas limites pour sécuriser les frontières du domaine métier, et des cas exceptionnels pour vérifier que les règles de validation sont correctement appliquées. C'est cette diversité qui permet d'obtenir une couverture réellement pertinente et de limiter les mauvaises surprises lors de l'évolution du code.

2. Intégration des tests dans le workflow de développement

Écrire des tests est une première étape importante, mais leur véritable valeur apparaît lorsqu'ils sont exécutés régulièrement tout au long du cycle de développement. Une suite de tests lancée uniquement avant une mise en production perd une grande partie de son intérêt. Dans un contexte professionnel, les tests doivent accompagner chaque évolution du projet afin de détecter les régressions le plus tôt possible.

L'objectif est donc d'intégrer les tests à plusieurs niveaux du workflow. Dès le développement local, un développeur peut exécuter les tests avant chaque commit ou chaque push. Lors des revues de code, un pipeline d'intégration continue prend généralement le relais pour vérifier automatiquement que les modifications n'introduisent aucun dysfonctionnement. Enfin, des outils complémentaires permettent de suivre la qualité globale du projet grâce aux rapports de couverture ou à la surveillance des dépréciations.

Cette approche permet de détecter les problèmes au moment où ils sont introduits, plutôt qu'après plusieurs jours ou semaines de développement.

Dans le cas de Symfony, il est recommandé d'exécuter les tests sur les serveurs d'intégration continue avec le mode debug désactivé. Cette configuration améliore les performances et se rapproche davantage des conditions réelles d'exécution. Lorsque l'environnement de test n'est pas entièrement recréé à chaque exécution, il peut également être nécessaire de gérer explicitement le cache dans tests/bootstrap.php afin d'éviter certains comportements inattendus.

L'intégration continue repose généralement sur une pipeline automatisée. Que l'on utilise GitLab CI/CD, GitHub Actions ou Jenkins, le principe reste similaire : préparer un environnement, installer les dépendances puis exécuter la suite de tests.

Dans GitLab CI/CD, un pipeline minimal peut ressembler à ceci :

```
image: php:8.4

before_script:
  - php -v
  - composer install --no-interaction --prefer-dist

test:
  script:
    - php bin/phpunit
```

À chaque exécution, GitLab crée un environnement propre, installe les dépendances du projet puis lance PHPUnit. Si un test échoue, le pipeline est immédiatement signalé comme invalide.

Cette automatisation devient encore plus intéressante lorsqu'une application doit fonctionner sur plusieurs versions de PHP. GitLab permet par exemple de dupliquer les jobs en utilisant différentes images Docker afin de vérifier la compatibilité du projet sur plusieurs environnements. De la même manière, des services complémentaires comme MySQL, PostgreSQL ou Redis peuvent être démarrés automatiquement lorsque les tests en ont besoin.

Au-delà de la simple réussite des tests, il est souvent utile de mesurer quelles parties du code sont réellement couvertes par la suite de tests. C'est le rôle des rapports de couverture.

PHPUnit peut générer différents types de rapports :

- Couverture de ligne
- Couverture de branche
- Couverture de chemin

Tous droits réservés à STUDI - Reproduction interdite

- Couverture de fonction
- Couverture de classe

Ces métriques permettent d'identifier les portions du code qui ne sont jamais exécutées pendant les tests. Pour produire ces rapports, PHPUnit doit s'appuyer sur un moteur de couverture. Les plus courants sont Xdebug et PCOV.

Le choix du moteur dépend du niveau de détail recherché, mais Xdebug reste la solution la plus complète lorsqu'une analyse approfondie de la couverture est nécessaire.

Dans certains cas, il est également pertinent d'indiquer explicitement quelle classe est couverte par un test. PHPUnit fournit pour cela différents attributs.

Par exemple :

```
#[CoversClass(PriceCalculator::class)]
```

Cet attribut permet de lier clairement le test à l'unité de code concernée et d'améliorer la précision des rapports de couverture. À l'inverse, certains tests plus globaux n'ont pas vocation à participer au calcul de couverture.

Dans ce cas, l'attribut suivant peut être utilisé :

```
#[CoversNothing]
```

Cette approche évite qu'un test d'intégration ou un scénario applicatif complexe fausse les statistiques de couverture des véritables tests unitaires.

Enfin, dans l'écosystème Symfony, le PHPUnit Bridge apporte une fonctionnalité particulièrement intéressante pour les projets qui évoluent sur plusieurs années : la gestion des dépréciations. Grâce à l'outil simple-phpunit, il devient possible d'obtenir un rapport spécifique recensant les fonctionnalités obsolètes utilisées par le projet. Cette information est précieuse lors des montées de version du framework.

L'utilisation de SYMFONY_DEPRECATIONS_HELPER permet notamment de générer une baseline contenant les dépréciations déjà connues. Les futures exécutions mettent alors uniquement en évidence les nouvelles alertes, ce qui évite de noyer les développeurs sous une masse d'informations déjà identifiées.

3. Exercice pratique : tester une exception PHP

Après avoir découvert les tests d'exceptions, il est temps de mettre en pratique cette notion avec un exercice simple mais très représentatif des besoins rencontrés dans les applications réelles. L'objectif est de vérifier qu'une méthode refuse correctement une donnée invalide en déclenchant une exception adaptée.

On considère ici une méthode permettant de définir le stock d'un produit :

```
public function setStock(int $stock): void
```

Dans notre contexte métier, un stock négatif n'a aucun sens. La méthode doit donc protéger cette règle en lançant une InvalidArgumentException lorsque la valeur transmise est inférieure à zéro. L'objectif du test consiste à vérifier que cette règle est bien respectée.

Pour cela, PHPUnit permet de déclarer explicitement l'exception attendue avant l'exécution de l'instruction concernée.

Voici un exemple complet :

```php
<?php

namespace App\Tests\Entity;

use App\Entity\Product;
use PHPUnit\Framework\TestCase;

final class ProductTest extends TestCase
```

Tous droits réservés à STUDI - Reproduction interdite

```
{
    public function testSetStockThrowsExceptionWhenStockIsNegative(): void
    {
        $product = new Product();

        $this->expectException(\InvalidArgumentException::class);
        $this->expectExceptionMessage(
            'Le stock ne peut pas être négatif.'
        );

        $product->setStock(-5);
    }
}
```

Au-delà de l'aspect technique, cet exercice illustre une notion essentielle : une exception fait partie intégrante du contrat d'une méthode. Lorsque l'on documente mentalement le comportement de setStock(), on ne doit pas seulement retenir qu'elle accepte des valeurs positives. Il faut également préciser ce qu'elle fait lorsqu'une valeur invalide est fournie.

Tester uniquement les cas favorables laisserait une partie importante du comportement non vérifiée. À l'inverse, un test d'exception garantit que la règle métier continue d'être appliquée même après une évolution du code ou une refactorisation.

À la fin de cet exercice, plusieurs points doivent être validés :

- L'exception attendue est correctement déclarée,
- Le message associé est vérifié,
- Les attentes sont placées avant l'instruction concernée,
- Le scénario métier d'erreur est couvert,
- Le test s'exécute avec succès dans PHPUnit.

Même si ce scénario paraît simple, il reflète une pratique essentielle du développement professionnel : considérer les comportements d'erreur comme des fonctionnalités à part entière et leur accorder le même niveau de validation que les scénarios fonctionnels classiques.

Tous droits réservés à STUDI - Reproduction interdite

IV Synthèse

A. Flashcards

Test unitaire

Vérifie le comportement d'une unité de code isolée en comparant résultat obtenu et attendu.

PHPUnit

Framework de référence pour écrire et exécuter des tests unitaires en PHP.

Pattern AAA

Structure de test en trois étapes : Arrange, Act, Assert.

Given/When/Then

Pattern BDD décrivant contexte initial, action déclencheur et résultat attendu.

Méthode setUp()

Exécutée automatiquement avant chaque test pour préparer l'environnement nécessaire.

assertSame()

Vérifie strictement valeur et type, plus strict que assertEquals().

expectException()

Déclare l'exception attendue avant l'instruction censée la déclencher.

Cas limites

Scénarios aux frontières du domaine métier, souvent sources de bugs.

TDD

Test Driven Development : écrire le test avant le code de production.

Couverture de code

Mesure les portions du code exécutées pendant les tests via Xdebug ou PCOV.

PHPUnit Bridge

Composant Symfony enrichissant PHPUnit, notamment pour gérer les dépréciations.

Indépendance des tests

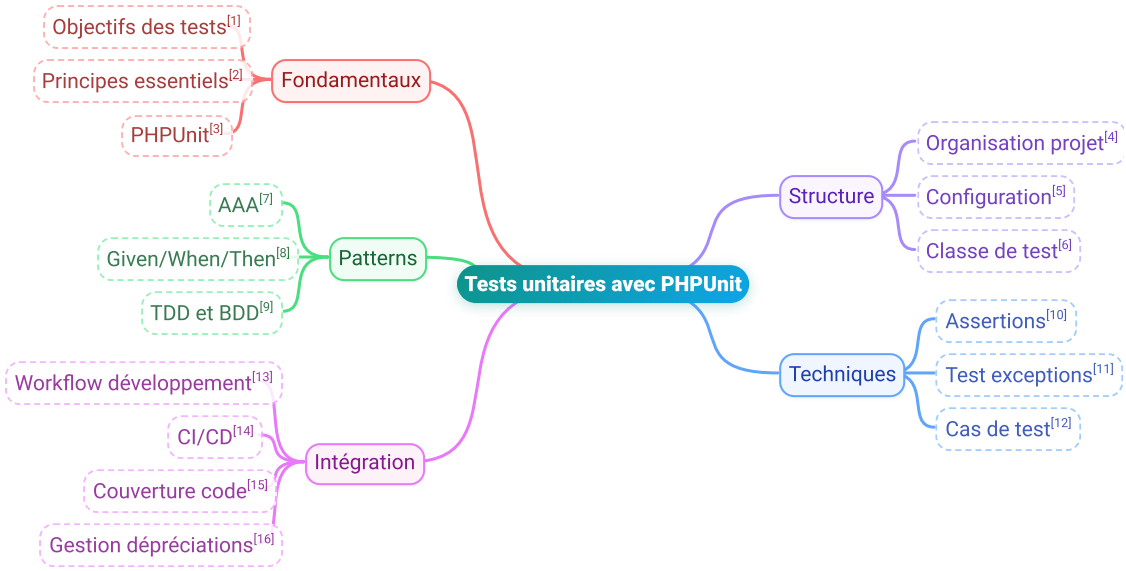
Chaque test s'exécute seul sans dépendre d'un autre ou d'un ordre particulier.

La théorie en bref

Fondamental

Les **tests unitaires** constituent une pratique essentielle du développement professionnel pour garantir la **qualité du code** et prévenir les **régressions**. **PHPUnit** est le framework de référence dans l'écosystème PHP, intégré nativement dans Symfony via le package **symfony/test-pack**. Un test unitaire vérifie le comportement d'une **unité de code isolée** en comparant le résultat obtenu au résultat attendu, selon trois principes fondamentaux : **rapidité**, **déterminisme** et **indépendance**. La structure d'un test suit généralement le pattern **AAA** (Arrange, Act, Assert) ou **Given/When/Then**, permettant de séparer clairement la préparation, l'exécution et la vérification. Les tests doivent couvrir les **cas normaux**, les **cas limites** et les **cas exceptionnels**, notamment en vérifiant le déclenchement des **exceptions** attendues via les méthodes expectException() de PHPUnit.

Tous droits réservés à STUDI - Reproduction interdite



1. Détecter les anomalies tôt, servir de filet de sécurité, documenter le comportement attendu.
2. Rapidité d'exécution, déterminisme des résultats, indépendance des scénarios.
3. Framework de référence pour les tests unitaires en PHP, intégré dans Symfony.
4. Dossier tests/ miroir de src/, séparation Unit/Integration/Application.
5. Fichiers phpunit.dist.xml, tests/bootstrap.php, environnement test.
6. Hérite de TestCase, méthodes setUp() et tearDown(), namespace cohérent.
7. Arrange (préparer), Act (exécuter), Assert (vérifier).
8. Contexte initial, action déclencheur, résultat attendu.
9. Test Driven Development et Behavior Driven Development.
10. assertEquals, assertSame, assertTrue pour vérifier les résultats.
11. expectException(), expectExceptionMessage() avant l'instruction concernée.
12. Couvrir cas normaux, cas limites et cas exceptionnels.
13. Exécution locale, revues de code, intégration continue.
14. Pipeline automatisée GitLab CI, GitHub Actions, Jenkins.
15. Rapports Xdebug ou PCOV, attributs CoversClass et CoversNothing.
16. PHPUnit Bridge, SYMFONY_DEPRECATIONS_HELPER, baseline.

Tests unitaires avec PHPUnit

Les conseils pro

Conseil

Comment structurer efficacement une suite de tests unitaires dans un projet Symfony ?

- Organiser le dossier tests/ en miroir du dossier src/ pour faciliter la navigation.
- Séparer les tests unitaires, d'intégration et applicatifs dans des sous-dossiers dédiés.
- Adopter une convention de nommage stricte : ClasseTest.php pour chaque Classe.php.
- Vérifier que chaque test peut s'exécuter indépendamment sans ordre imposé.

Comment garantir la qualité et la maintenabilité des tests unitaires ?

Tous droits réservés à STUDI - Reproduction interdite

- Appliquer systématiquement le pattern AAA ou Given/When/Then pour structurer chaque test.
- Rédiger des noms de méthodes explicites décrivant précisément le scénario testé.
- Limiter chaque test à une seule assertion métier pour faciliter le diagnostic d'échec.
- Utiliser setUp() pour factoriser la préparation commune à plusieurs tests.

Comment couvrir efficacement les différents scénarios métier dans les tests ?

- Identifier et tester en priorité les cas normaux correspondant aux usages courants.
- Couvrir systématiquement les cas limites : valeurs nulles, zéro, maximales, frontières.
- Vérifier les cas exceptionnels en testant le déclenchement des exceptions attendues.
- Utiliser des jeux de données variés pour valider la robustesse de la logique métier.

Comment intégrer les tests unitaires dans le workflow de développement ?

- Exécuter les tests localement avant chaque commit pour détecter les régressions immédiatement.
- Configurer un pipeline CI/CD automatisant l'exécution des tests à chaque push.
- Générer régulièrement des rapports de couverture pour identifier les zones non testées.
- Exploiter le PHPUnit Bridge pour surveiller et traiter les dépréciations Symfony.

Comment tester efficacement les exceptions dans PHPUnit ?

- Déclarer expectException() juste avant l'instruction censée déclencher l'erreur.
- Compléter avec expectExceptionMessage() pour vérifier le message d'erreur attendu.
- Considérer les exceptions comme partie intégrante du contrat métier à valider.
- Tester systématiquement les validations métier refusant les données invalides.

Tous droits réservés à STUDI - Reproduction interdite

V Auto-évaluation

A. Exercice

Cas pratique

Vous développez une plateforme e-commerce pour un réseau de boutiques françaises. Un service central, PriceCalculatorService, est chargé de calculer le prix total d'un panier en tenant compte des remises, de la TVA et des frais de livraison.

Question

[solution n°1 p. 27]

Concevez une stratégie de tests unitaires pour ce service. Expliquez quelles méthodes vous testeriez en priorité, quels scénarios (cas normaux, cas limites, cas exceptionnels) vous couvririez, comment vous isoleriez les dépendances externes du service, et quelle structure de tests vous mettriez en place avec PHPUnit pour garantir la fiabilité et la maintenabilité de la suite de tests.

B. Test

Exercice 1 : Quiz

[solution n°2 p. 28]

Question 1

Quel est l'objectif principal d'un test unitaire ?

- Vérifier le fonctionnement d'une base de données
- Tester une unité de code de manière isolée
- Déployer une application en production
- Mesurer les performances du serveur

Question 2

Quel framework est le plus utilisé pour réaliser des tests unitaires en PHP ?

- Doctrine
- Twig
- PHPUnit
- Composer

Question 3

Quelle méthode PHPUnit permet de vérifier que deux valeurs sont égales ?

- assertEquals()

Tous droits réservés à STUDI - Reproduction interdite

assertNull()

assertTrue()

assertInstanceOf()

Question 4

Pourquoi est-il recommandé d'isoler les dépendances lors d'un test unitaire ?

Pour accélérer le navigateur

Pour éviter que le test dépende de services externes ou de ressources réelles

Pour remplacer Composer

Pour générer automatiquement la documentation

Question 5

Quelle est une bonne pratique lors de l'écriture d'un test unitaire ?

Tester plusieurs comportements différents dans un seul test

Utiliser des noms de méthodes explicites

Dépendre d'une connexion Internet

Modifier la base de données de production

Question 6

Quel est l'un des pièges les plus fréquents lors de l'écriture de tests unitaires ?

Tester uniquement les cas où tout se passe bien

Utiliser des méthodes d'assertion adaptées

Exécuter les tests après une modification du code

Donner un nom explicite aux méthodes de test

Exercice 8

[solution n°3 p. 30]

Associez chaque scénario de test à l'approche ou à l'assertion la plus adaptée.

Tous droits réservés à STUDI - Reproduction interdite

Vérifier qu’une condition est vraie Vérifier qu’une méthode retourne la valeur attendue Tester une classe dépendant d’un service externe Vérifier qu’une exception est levée	expectException() Utilisation d’un mock assertTrue() assertEquals()

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

[exercice p. 24] Solution n°1

Pour concevoir une stratégie de test unitaire pour un service de calcul de prix dans une plateforme e-commerce, l'objectif serait de vérifier que chaque règle métier est correctement appliquée : remises, taxes, frais de livraison, cas particuliers et erreurs possibles. Ce service étant central dans l'expérience d'achat, les tests doivent être précis, lisibles et couvrir les scénarios les plus sensibles.

La première étape consisterait à identifier les méthodes importantes à tester en priorité. On testerait notamment la méthode principale de calcul du prix total, par exemple calculateTotalPrice(), car elle regroupe les différentes règles métier. On testerait aussi les méthodes liées au calcul des remises, des taxes, des frais de livraison, ainsi que les éventuelles méthodes de validation des données d'entrée. L'objectif est de s'assurer que le service retourne toujours un prix juste, cohérent et conforme aux règles commerciales.

Les cas normaux seraient les premiers à être couverts. Par exemple, un panier contenant plusieurs produits avec un prix unitaire, une quantité, une TVA standard et des frais de livraison classiques. Le test vérifierait que le sous-total est correctement calculé, que la TVA est bien appliquée et que les frais de livraison sont ajoutés au montant final. On pourrait également tester un panier avec une remise simple, par exemple 10 % à partir d'un certain montant.

Les cas limites seraient également essentiels. Il faudrait tester un panier vide, un produit avec une quantité égale à zéro, un prix très faible, un prix très élevé, ou encore un montant situé exactement au seuil d'activation d'une remise ou de la livraison gratuite. Par exemple, si la livraison devient gratuite à partir de 100 €, il faudrait tester un panier à 99,99 €, à 100 € et à 100,01 € afin de vérifier que la règle s'applique au bon moment.

Les cas exceptionnels permettraient de vérifier que le service réagit correctement face à des données invalides. Par exemple, un prix négatif, une quantité négative, une taxe inconnue, un pays de livraison non pris en charge ou une remise invalide. Dans ces situations, le service devrait lever une exception claire, comme InvalidArgumentException ou DomainException, afin d'éviter de produire un résultat incohérent.

Pour garder de vrais tests unitaires, il serait important d'isoler le service de ses dépendances. Le service de calcul ne devrait pas dépendre directement de la base de données, d'un service d'authentification ou d'une API externe. Si ces dépendances sont nécessaires, elles seraient remplacées par des mocks ou des stubs. Par exemple, un TaxProviderInterface pourrait être mocké pour retourner un taux de TVA précis, un ShippingCostProviderInterface pourrait simuler différents frais de livraison, et un DiscountPolicyInterface pourrait retourner une remise connue. Cette approche permet de tester uniquement la logique du service de prix, sans être impacté par l'état de la base de données ou par un service externe.

Une structure de test claire pourrait être mise en place avec PHPUnit. La classe de test pourrait s'appeler PriceCalculatorServiceTest. Chaque méthode de test devrait porter un nom explicite afin de décrire le comportement attendu.

Tous droits réservés à STUDI - Reproduction interdite

[exercice p. 24]

Solution n°2

Question 1

Quel est l'objectif principal d'un test unitaire ?

- Vérifier le fonctionnement d'une base de données
- ✓ Tester une unité de code de manière isolée
- Déployer une application en production
- Mesurer les performances du serveur

Un test unitaire vérifie le comportement d'une méthode ou d'une classe de manière isolée afin de s'assurer qu'elle produit le résultat attendu.

Question 2

Quel framework est le plus utilisé pour réaliser des tests unitaires en PHP ?

- Doctrine
- Twig
- ✓ PHPUnit
- Composer

PHPUnit est le framework de référence pour écrire, organiser et exécuter des tests unitaires dans les projets PHP.

Question 3

Quelle méthode PHPUnit permet de vérifier que deux valeurs sont égales ?

- ✓ assertEquals()
- assertNull()
- assertTrue()
- assertInstanceOf()

Tous droits réservés à STUDI - Reproduction interdite

La méthode assertEquals() compare une valeur attendue avec une valeur obtenue afin de vérifier leur égalité.

Question 4

Pourquoi est-il recommandé d'isoler les dépendances lors d'un test unitaire ?

Pour accélérer le navigateur

✓ Pour éviter que le test dépende de services externes ou de ressources réelles

Pour remplacer Composer

Pour générer automatiquement la documentation

Les tests unitaires doivent vérifier uniquement le comportement du code testé. L'utilisation de mocks ou de stubs permet d'éviter les dépendances à une base de données, une API ou un service externe.

Question 5

Quelle est une bonne pratique lors de l'écriture d'un test unitaire ?

Tester plusieurs comportements différents dans un seul test

✓ Utiliser des noms de méthodes explicites

Dépendre d'une connexion Internet

Modifier la base de données de production

Des noms de tests explicites facilitent la compréhension, la maintenance et le diagnostic des éventuelles erreurs.

Question 6

Quel est l'un des pièges les plus fréquents lors de l'écriture de tests unitaires ?

✓ Tester uniquement les cas où tout se passe bien

Utiliser des méthodes d'assertion adaptées

Exécuter les tests après une modification du code

Tous droits réservés à STUDI - Reproduction interdite

Donner un nom explicite aux méthodes de test

Tester uniquement les cas nominaux peut laisser passer de nombreuses erreurs. Il est important de prévoir également des cas limites, des valeurs inattendues et des situations d'échec afin de garantir la robustesse de l'application.

Solution n°3

[exercice p. 25]

Associez chaque scénario de test à l'approche ou à l'assertion la plus adaptée.

expectException() Vérifier qu'une exception est levée	**Utilisation d'un mock** Tester une classe dépendant d'un service externe
assertTrue() Vérifier qu'une condition est vraie	**assertEquals()** Vérifier qu'une méthode retourne la valeur attendue

- assertEquals() permet de comparer le résultat obtenu avec la valeur attendue.
- expectException() vérifie qu'une exception spécifique est bien déclenchée.
- Un mock permet de simuler le comportement d'une dépendance afin d'isoler le code testé.
- assertTrue() permet de vérifier qu'une expression retourne la valeur true.

Tous droits réservés à STUDI - Reproduction interdite